CSE346 Business Process Modeling

Assignment 2: Process Mining and Automation

Alpha-algorithm computation, PERT/event-network evidence, and Python automation

Submission focus

This report explains the Alpha algorithm, shows the exact computations from the event log, converts the mined relations into a PERT/event-network view, and documents a reusable Python automation workflow.

Team Members

Name	Code
Seif Eldeen Ibrahim	22P0023
Anas Bayoumi	22P0011
Nadine Radwan	22P0214
Jana Sameh	22P0237

Contents

List of Figures

1 Executive Scope

The assignment has two required tasks. Task One mines the provided event log using the Alpha algorithm and translates the mined relations into a PERT/event-network chart. Task Two automates the same work from a structured CSV file. The implementation uses Python because it can parse timestamps, group cases, compute relations exactly, generate reusable evidence files, and rebuild the report tables without manual spreadsheet copying.

2 Evaluation Criteria Coverage

The assignment rubric is used as an explicit checklist. The report covers each marking category as follows:

Criteria category	Marks	How this report covers it
Alpha algorithm application	15	The report states the Alpha definitions, builds T_W, T_I, T_O, direct succession, causality, parallelism, choice, candidate places, maximal places, source/sink places, and exact counts. Intermediate relations are included in generated tables.
Event network / PERT chart	15	The mined causal pairs are translated into a PERT/event-network chart. The Mermaid and DOT sources are included, and the SVG export is rendered into the PDF with readable labels.
Process understanding and interpretation	10	The interpretation discusses stock/manufacturing choices, convergence at confirmation, invoice/address concurrency, payment/shipment concurrency, archive readiness, and the Case 3 event-ID anomaly.
Clarity of documentation	10	The report includes a cover page, contents, list of figures, preprocessing explanation, exact computation tables, generated visual evidence, command evidence, and clear rerun instructions.
Tool selection and justification	5	Python is justified because it supports CSV import, timestamp parsing, grouping by case, deterministic relation computation, generated evidence files, and reuse on new logs.
Data preprocessing	10	The event log is structured into `case_id`, `timestamp`, `activity`, and `resource`; events are sorted by timestamp per case; the Case 3 label/prefix inconsistency is documented and handled.
Alpha algorithm automation	15	`tools/process_mining_alpha.py` implements the Alpha logic and outputs traces, relation counts, maximal places, timing metrics, Mermaid, DOT, CSV, and text summaries.
Generation of PERT chart	10	The automation writes `output/assignment2_pert.mmd` and `output/assignment2_alpha_net.dot`; the visual chart is validated against the same causal pairs used in Task One.
Usability and reusability	10	The script accepts an input CSV and output directory from the command line, has a stable input schema, and can be rerun on a different event log with the same fields.

3 Lecture Concepts and Algorithm

The process-mining lecture defines the minimum event-log fields as case IDs and task IDs, with optional timestamp, resource, and data fields. This assignment uses case ID, timestamp, activity, and resource.

The Alpha relations used in the computation are:

- Direct succession: $x > y$ when x is directly followed by y in at least one case.
- Causality: $x \to y$ when $x > y$ and not $y > x$.
- Parallelism: $x \parallel y$ when both $x > y$ and $y > x$.
- Choice: $x \# y$ when neither $x > y$ nor $y > x$.

The implemented Alpha algorithm follows these steps:

1. Build T_W, the set of all activities in the workflow log.
2. Build T_I, the activities that appear first in at least one trace.
3. Build T_O, the activities that appear last in at least one trace.
4. Compute direct succession, causality, parallelism, and choice.
5. Build candidate pairs (A, B) where every $a \in A$ causes every $b \in B$, activities inside A are pairwise choice, and activities inside B are pairwise choice.
6. Keep only maximal pairs Y_W and convert them into Alpha places.
7. Add source and sink places to produce the mined workflow net.

4 Preprocessing

The raw assignment text was converted into `data/event_log.csv`, one row per event. Events are sorted by timestamp inside each case. One naming inconsistency in Case 3 was handled by trusting the activity label instead of the event ID prefix: the event ID beginning with `Sh` is labeled "Get shipping address", and the event ID beginning with `Ge` is labeled "Ship product". This preserves the activity sequence stated in the assignment text.

Preprocessing item	Handling
Case identification	Events are grouped by `case_id`; four cases are present.
Timestamp handling	Timestamps are parsed with the fixed format `YYYY-MM-DD HH:MM` and sorted inside each case before mining.
Activity labels	Full activity labels are preserved for computation; shorter names are used only for report readability and diagrams.
Resource field	Resources such as SYS1, SYS2, DMS, and employees are retained in the cleaned CSV for handoff analysis.
Quality issue	Case 3 has two event IDs whose prefixes do not match their labels; the activity labels are treated as authoritative because the assignment text defines the actual task sequence.

5 Generated Alpha-Algorithm Evidence

5.1 Traces

Case	Trace
1	Check stock → Retrieve item → Confirm order → Get address → Emit invoice → Receive payment → Ship product → Archive order
2	Check stock → Check materials → Request materials → Obtain materials → Manufacture → Confirm order → Emit invoice → Get address → Ship product → Receive payment → Archive order
3	Check stock → Check materials → Manufacture → Confirm order → Emit invoice → Get address → Ship product → Receive payment → Archive order
4	Check stock → Retrieve item → Confirm order → Get address → Emit invoice → Receive payment → Ship product → Archive order

5.2 Exact Computation Totals

Computed item	Value
Cases in the log	4
Events in the log	36
Activities in T_W	12
Start activities in T_I	1
End activities in T_O	1
Observed direct-succession pairs $>$	18
Causal pairs $\rightarrow$	14
Parallel ordered pairs $\|$	4
Parallel unordered pairs	2
Choice ordered pairs $\#$	100
Maximal Alpha places Y_W	11
Average cycle time	5.28 days

5.3 Direct Succession and Causality

From	To	Count	Alpha relation
Check materials	Manufacture	1	→
Check materials	Request materials	1	→
Check stock	Check materials	2	→
Check stock	Retrieve item	2	→
Confirm order	Emit invoice	2	→
Confirm order	Get address	2	→
Emit invoice	Get address	2	‖
Emit invoice	Receive payment	2	→
Get address	Emit invoice	2	‖
Get address	Ship product	2	→
Manufacture	Confirm order	2	→
Obtain materials	Manufacture	1	→
Receive payment	Archive order	2	→
Receive payment	Ship product	2	‖

Request materials	Obtain materials	1	→
Retrieve item	Confirm order	2	→
Ship product	Archive order	2	→
Ship product	Receive payment	2	‖

5.4 Alpha Sets

- T_W activities: Archive order, Check materials, Check stock, Confirm order, Emit invoice, Get address, Manufacture, Obtain materials, Receive payment, Request materials, Retrieve item, Ship product
- T_I start activities: Check stock
- T_O end activities: Archive order
- Parallel pairs: (Emit invoice, Get address), (Receive payment, Ship product)
- Main exclusive choices: available stock path versus manufacturing path; raw materials already available versus raw materials requested and obtained.

5.5 Maximal Alpha Places

Input activity set A	**Output activity set B**
{Check materials}	{Manufacture, Request materials}
{Check materials, Obtain materials}	{Manufacture}
{Check stock}	{Check materials, Retrieve item}
{Confirm order}	{Emit invoice}
{Confirm order}	{Get address}
{Emit invoice}	{Receive payment}
{Get address}	{Ship product}
{Manufacture, Retrieve item}	{Confirm order}
{Receive payment}	{Archive order}
{Request materials}	{Obtain materials}
{Ship product}	{Archive order}

5.6 Timing Metrics

Case	Start	End	Cycle time
1	2012-07-30 11:14	2012-08-06 09:12	6.92 days
2	2012-08-01 09:44	2012-08-07 08:50	5.96 days
3	2012-08-02 10:06	2012-08-06 10:25	4.01 days
4	2012-08-04 08:11	2012-08-08 14:08	4.25 days

From	To	Occurrences	Average hours
Ship product	Receive payment	2	76.28
Emit invoice	Receive payment	2	57.32
Get address	Emit invoice	2	46.76
Request materials	Obtain materials	1	44.63

Obtain materials	Manufacture	1	30.88
Receive payment	Ship product	2	14.42
Ship product	Archive order	2	11.22
Check materials	Manufacture	1	5.88
Check stock	Retrieve item	2	3.46
Check materials	Request materials	1	1.10
Get address	Ship product	2	1.07
Retrieve item	Confirm order	2	0.47
Confirm order	Emit invoice	2	0.42
Confirm order	Get address	2	0.32
Check stock	Check materials	2	0.22
Emit invoice	Get address	2	0.20
Manufacture	Confirm order	2	0.13
Receive payment	Archive order	2	0.12

6 Mined Process Interpretation

The mined process has one start activity, *Check stock availability*, and one end activity, *Archive order*. The exact computations show 4 cases, 36 events, 12 activities, 18 observed direct-succession pairs, 14 causal pairs, 2 unordered parallel pairs, and 11 maximal Alpha places.

The behavior discovered from the traces is:

- After stock checking, the process chooses between retrieving an available warehouse item and entering the materials/manufacturing branch.
- In the manufacturing branch, checking materials may lead directly to manufacturing or to requesting and obtaining raw materials first.
- Warehouse retrieval and manufacturing converge at confirming the order.
- Invoice/address order differs across traces, so the Alpha algorithm detects *Emit invoice* and *Get shipping address* as parallel.
- Payment/shipment order also differs across traces, so *Receive payment* and *Ship product* are parallel.
- Archiving is possible only after payment and shipment completion evidence exists in the log.

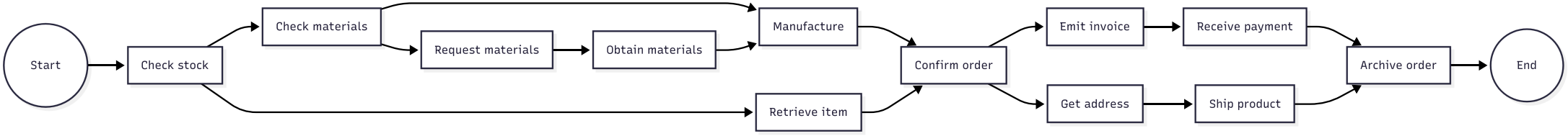
The log does not show a repeated-activity loop inside a case, so the mined model focuses on choices and concurrency rather than loop behavior. The main anomaly is the Case 3 event-ID prefix mismatch documented in preprocessing; it is treated as a data-quality issue rather than a behavioral exception because the activity names remain consistent with the overall process.

7 PERT/Event-Network Chart

The generated event network is stored in Mermaid and DOT so it can be opened in online tools rather than drawn in LaTeX:

- `output/assignment2_pert.mmd`
- `output/assignment2_alpha_net.dot`

The PERT dependencies are the causal pairs from the Alpha calculation. Start connects to *Check stock*; *Check stock* branches to either *Retrieve item* or *Check materials*; raw materials may be requested and obtained before manufacturing; *Manufacture* and *Retrieve item* converge at *Confirm order*; invoice/address and payment/shipment are represented as concurrent branches; *Archive order* is the terminal activity.


Start
Check stock
Check materials
Request materials
Obtain materials
Manufacture
Retrieve item
Confirm order
Emit invoice
Receive payment
Get address
Ship product
Archive order
End

8 Automation Design

The automation script is `tools/process_mining_alpha.py`. It performs the following reusable workflow:

1. Read any CSV containing `case_id`, `timestamp`, `activity`, and `resource`.
2. Sort events per case by timestamp.
3. Extract traces, start activities T_I, end activities T_O, and task set T_W.
4. Count direct succession pairs.
5. Derive causality, parallel, and choice using the lecture definitions.
6. Build maximal Alpha places (A, B).
7. Compute case cycle times and average waiting times between directly successive activities.
8. Generate documentation outputs and online-visualization files.

Run command:

```
python3 tools/process_mining_alpha.py \
  --input data/event_log.csv \
  --output-dir output
```

Generated file	Purpose
`output/event_log_sorted.csv`	Clean sorted event log for checking preprocessing.
`output/assignment2_generated_tables.tex`	Exact Alpha evidence tables included in this report.
`output/assignment2_pert.mmd`	Mermaid PERT chart for Mermaid Live Editor.
`output/assignment2_alpha_net.dot`	DOT dependency graph for Graphviz or diagrams.net.
`output/assignment2_summary.txt`	Plain-text summary of counts, relations, and maximal places.

8.1 Reusable Input Contract

Reuse requirement	Implementation
Input schema	Any CSV can be used if it contains `case_id`, `timestamp`, `activity`, and `resource`. An optional `event_id` column is read when present.
Configurable paths	The command-line options `-input` and `-output-dir` let the user run the script on another log without changing the code.
Transparent outputs	The script writes the sorted log, generated LaTeX tables, Mermaid chart, DOT chart, and text summary so intermediate steps can be checked.
Validation method	Counts in the summary must match the generated report tables; the PERT chart must use the same causal relations as the Alpha relation table.

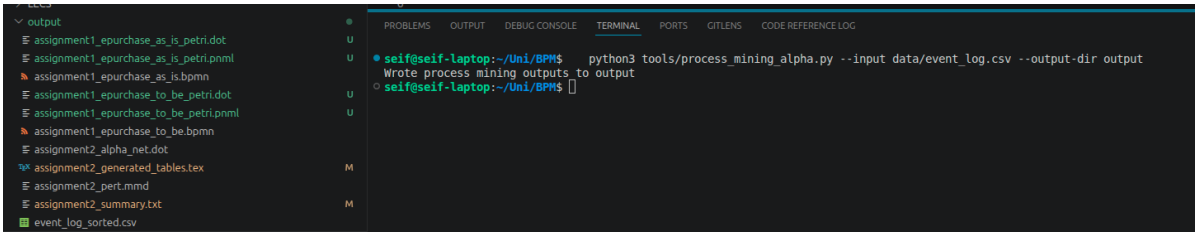


Figure 2. Automation run screenshot showing the Python script and generated output files.

9 Timing Findings and Improvements

The average observed cycle time is 5.28 days. The longest average waits are *Ship product* to *Receive payment* at 76.28 hours, *Emit invoice* to *Receive payment* at 57.32 hours, and *Get shipping address* to *Emit invoice* at 46.76 hours. These delays are waiting periods between handoffs, not the short system tasks themselves.

Recommended improvements are:

- Integrate payment and shipment status so archive readiness is visible immediately.
- Add alerts for long gaps after invoice emission or shipment dispatch.
- Standardize whether payment must precede shipment or whether both may proceed independently; the log shows both orders.
- Monitor resource handoffs between SYS1, SYS2, employees, and DMS because handoffs are common sources of delay and error.

10 Conclusion

The Alpha algorithm discovers an order-fulfillment process with stock/manufacturing choices and two concurrent sections. The automation produces the same evidence needed for Task One and reusable files for Task Two. The included PERT SVG and automation screenshot document the online visualization and script execution evidence.